

Chapter 1: Practical Session - Python Basics and Mathematical Libraries

Youssef SALMAN

September 2025

1. Introduction to Python and Scientific Computing

Python is a powerful and easy-to-learn programming language that is widely used in scientific computing, data analysis, and machine learning. Its readable syntax and large ecosystem of libraries make it an excellent choice for students and researchers.

1.1. What is Python?

Python is a general-purpose programming language that emphasizes code readability and simplicity. It supports multiple programming paradigms, including procedural, object-oriented, and functional programming.

Key features:

- Easy to read and write
- Large community and extensive documentation
- Extensive libraries for scientific computing (NumPy, SciPy, Pandas, Matplotlib, etc.)

1.2. Python for Scientific Computing

Python can be used interactively or as a scripting language. For scientific work, we often rely on environments that facilitate both coding and visualizing data.

Recommended environments:

- **Jupyter Notebook:** A web-based interactive environment that supports live code, equations, visualizations, and narrative text in a single document. Very useful for teaching and documentation.
- **Spyder:** An integrated development environment (IDE) similar to MATLAB, popular among scientists and engineers.
- **VS Code:** A general IDE with powerful Python support.

1.3. Python Program Structure

A typical Python program consists of:

- Importing libraries (e.g., `import numpy as np`)
- Defining variables
- Writing functions
- Performing calculations and printing results

Example:

```
import numpy as np

def square(x):
    return x**2

print(square(4)) # Output: 16
```

2. Basic Python Refresher

Python is a high-level programming language widely used in data science and machine learning.

2.1. Simple Operations and Variables

```
a = 5
b = 3
sum_result = a + b
print("Sum:", sum_result)
```

2.2. Loops and Functions

```
def factorial(n):
    result = 1
    for i in range(1, n+1):
        result *= i
    return result

print(factorial(5))
```

3. Introduction to NumPy

NumPy is the fundamental package for numerical computing in Python. It provides arrays and operations on them.

3.1. Array Creation and Basic Operations

```
import numpy as np

x = np.array([1, 2, 3])
y = np.array([4, 5, 6])

print(x + y)          # element-wise addition
print(x * 2)          # scalar multiplication
print(np.dot(x, y))   # dot product
```

4. Introduction to SciPy

SciPy builds on NumPy and provides more advanced scientific computing functions.

4.1. Using SciPy for Special Functions

```
from scipy import special

val = special.gamma(5) # gamma(n) = (n-1)!
print(val)
```

5. Exercises

The following exercises are designed to reinforce your understanding of Python programming, algebraic expressions, function definitions, and vector operations using NumPy and SciPy.

1. Write a Python function that evaluates the quadratic expression $f(x) = ax^2 + bx + c$ for a given x . Test it with $a = 2$, $b = 3$, $c = 1$, and $x = 4$.
2. Generate two NumPy arrays of size 4 representing vectors. Compute:
 - Their element-wise sum
 - Their scalar multiplication by 3
 - Their dot product
3. Define a list of numbers and write a Python function that returns the mean and variance (use NumPy functions).
4. Use SciPy to compute:
 - The factorial of 6 using `scipy.special.factorial`
 - The value of the Gamma function at 6.5
5. Define a Python function that checks if a given vector has zero mean. Use NumPy and test your function on a few examples.
6. **Real-world application:** In a simple recommendation system, users and items (e.g., movies) can be represented as vectors in a common feature space.
Suppose a user has preferences represented by a vector:

`user = [5, 2, 0, 1]`

and three items (movies) are described by:

`item1 = [4, 1, 0, 1], item2 = [1, 2, 3, 1], item3 = [5, 2, 1, 0]`

Compute the dot product between the user vector and each item. The item with the highest dot product is the most relevant recommendation.

Implement this in Python and explain briefly what the result represents.

6. Conclusion

In this session, we introduced the Python programming language and scientific development environments such as Jupyter and Spyder. We then reviewed Python syntax, demonstrated algebraic and vector operations with NumPy and SciPy, and concluded with applied exercises. These skills form the computational base for the rest of the course.